

COMP30026

Models of Computation

Lecture 18: Introduction to Turing Machines

Cezary Kaliszyk and William Umboh

Semester 2, 2026

Announcements

- A1 Consultation Hours tomorrow 9:30 – 11:30 am in MC Room 4207
- Cezary and I have office hours tomorrow as well
- Executable examples of automata on Ed Lessons
 - A2 will ask you to implement your solutions using same libraries
 - Use the ones in Ed Lessons as templates

Where are we?

Last few weeks:

Restricted models of computation

- Regular languages: Finite automata, regular expressions
- Context-free languages: Pushdown automata, and context-free grammars

$$\{0^n 1^n 2^n \mid n \geq 0\}.$$

Today: (Sipser §3.1 - §3.2)

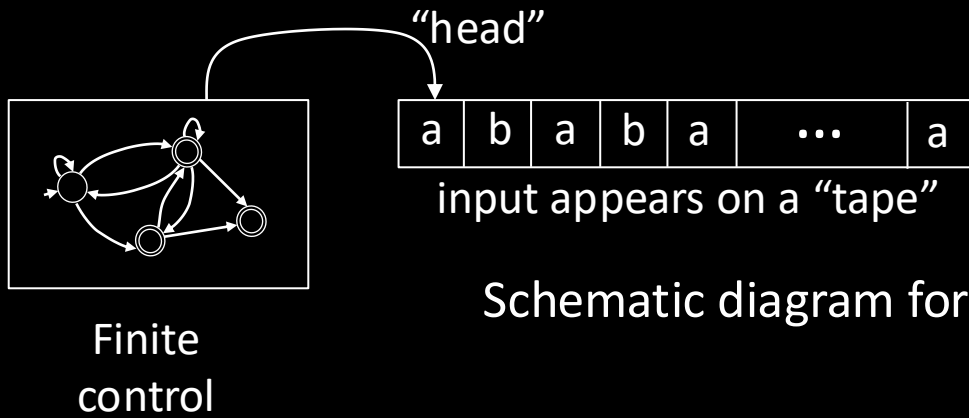
Turing machines (unrestricted model of computation)

- Turing-recognizable and Turing-decidable languages
- Church-Turing Thesis

Previously, on Models of Computation

Machine Model

Finite Automata



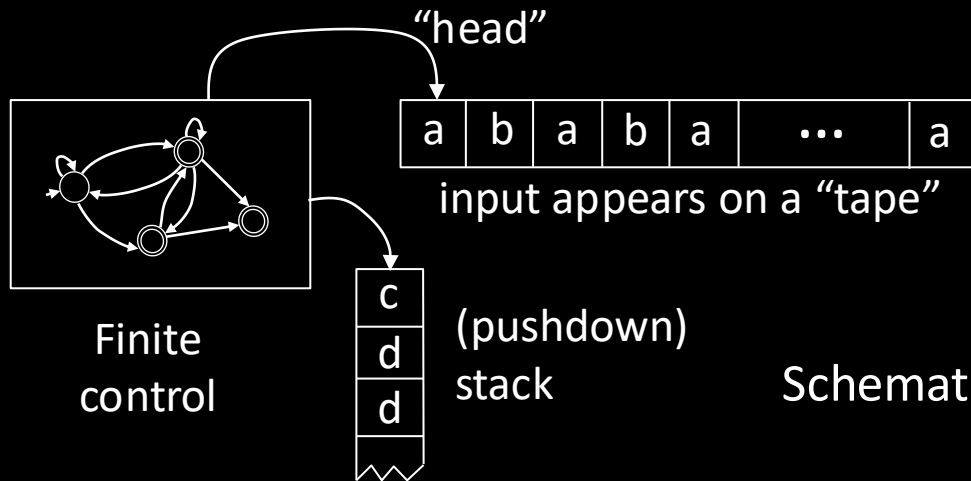
Schematic diagram for DFA or NFA

Note: "Memory" bounded by size of finite control

Previously, on Models of Computation

Machine Model

Pushdown Automata

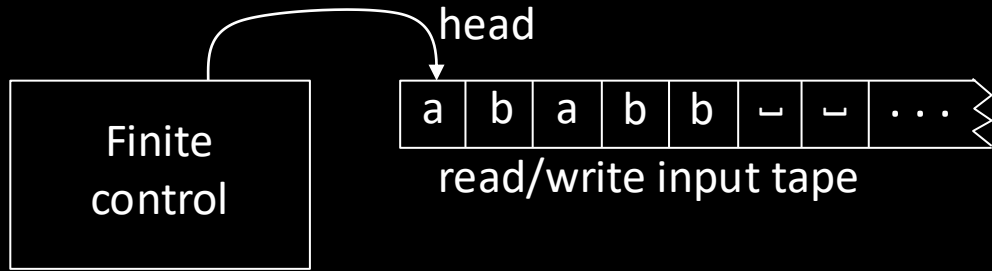


Schematic diagram for PDA

Operates like an NFA except can push/pop symbols to/from the top of stack.

Note: "Memory" unbounded by size of finite control but still restricted (stack access)

Turing Machines (TMs) - Informal

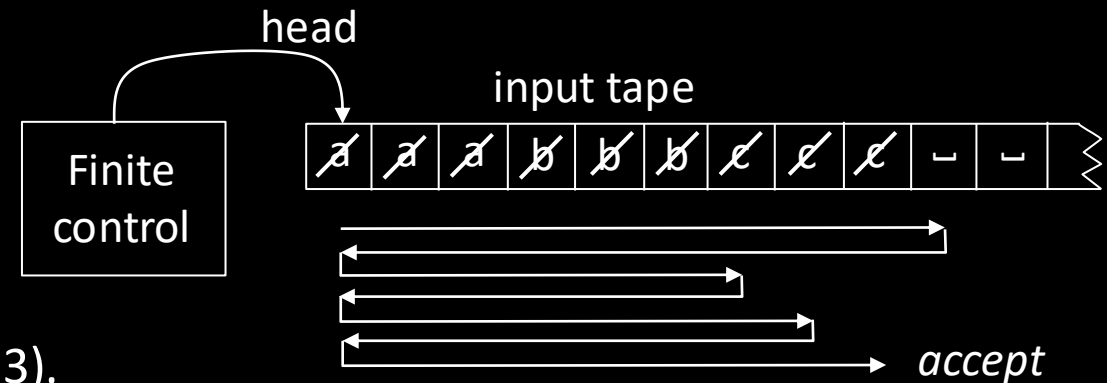


- 1) Head can read and write
- 2) Head is two way (can move left or right)
- 3) Tape is infinite (to the right)
- 4) Infinitely many blanks " $\sqcup$ " follow input
- 5) Can accept or reject any time (not only at end of input)

TM – Informal example

TM recognizing $B = \{a^k b^k c^k \mid k \geq 0\}$ (how to program this?)

- 1) Scan right until $\sqcup$ while checking if input is in $a^* b^* c^*$, *reject* if not.
- 2) Return head to left end.
- 3) Scan right, crossing off single a, b, and c.
- 4) If the last one of each symbol, *accept*.
- 5) If the last one of some symbol but not others, *reject*.
- 6) If all symbols remain, return to left end and repeat from (3).



How do we get the effect of “crossing off” with a Turing machine?

We use a tape alphabet $\Gamma = \{a, b, c, \cancel{a}, \cancel{b}, \cancel{c}, \sqcup\}$.

TM – Formal Definition

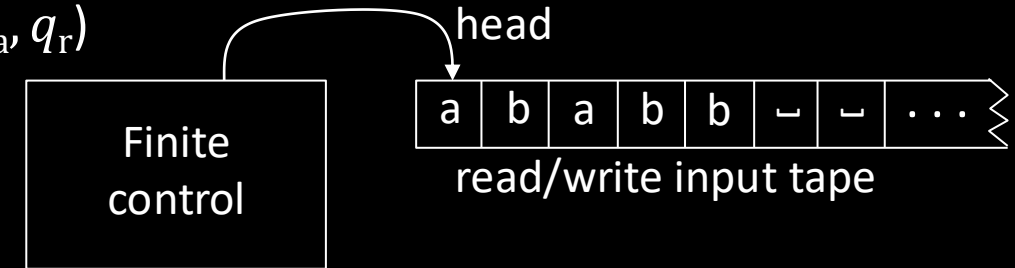
Defn: A Turing Machine (TM) consists of

Σ input alphabet

Γ tape alphabet ($\Sigma \subseteq \Gamma$) incl. blank character $\sqcup$

q_0 initial state, q_{acc} accept state, q_{rej} reject state (sometimes q_a, q_r)

$\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ (L = Left, R = Right)



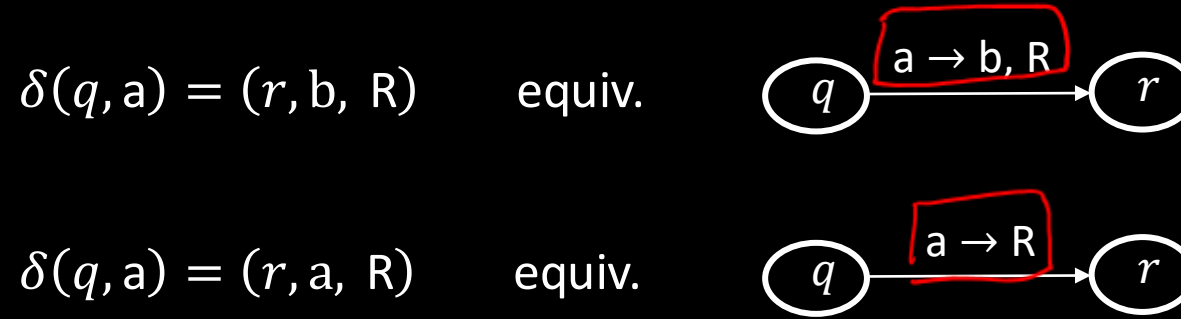
$$\delta(\underline{q}, \underline{a}) = (\underline{r}, \underline{b}, \underline{L})$$

If current state is q and current symbol under tape head is a ,

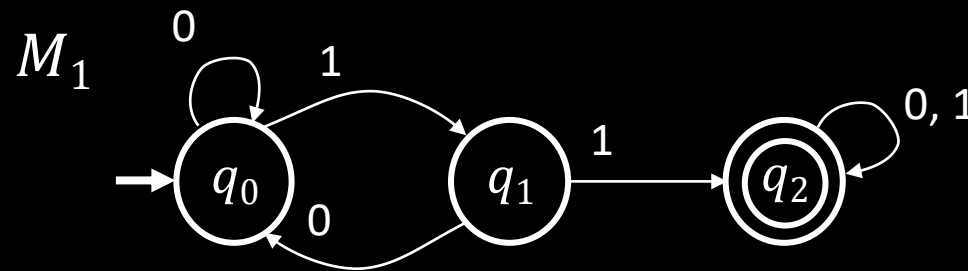
1. Change state to r
2. Over-write tape symbol a by b
3. Move the tape head to the left by one cell (tape head does not move if already at the start)

Drawing TMs

We can have a graphical notation for Turing Machines similar to that for finite automata:



Example The following DFA recognizes language $\{w \mid w \text{ contains substring } 11\}$



q_2 : w contains 11

q_1 : w does not contain 11 and ends with 1

q_0 : w empty or w does not contain 11 and ends with 0

IMPORTANT Keep in mind the “meaning” of states: information about string so far

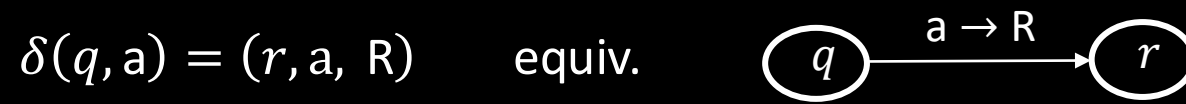
What do strings with resulting state q_i have in common?

Aside: Tip for Designing FAs

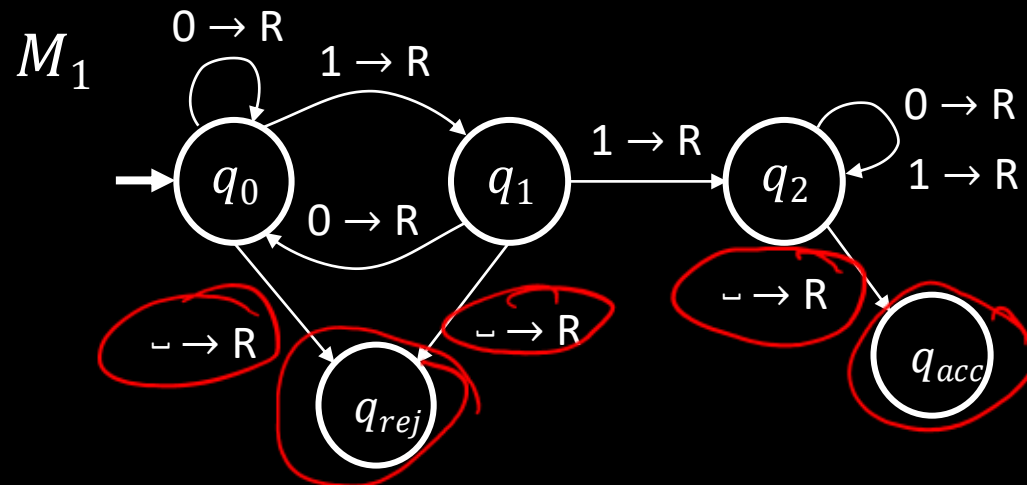
- Do not start by drawing random state diagrams
- Think about what information about the current string you need to maintain
- Write a program (in your favorite language) to recognize language with the following restrictions:
 - No new variables at run-time
 - Input symbols given to program one at a time (Single-digit tally counter example in Ed Lessons)

Drawing TMs

We can have a graphical notation for Turing Machines similar to that for finite automata:



Example The following TM recognizes language $\{w \mid w \text{ contains substring } 11\}$



q_2 : w contains 11

q_1 : w does not contain 11 and ends with 1

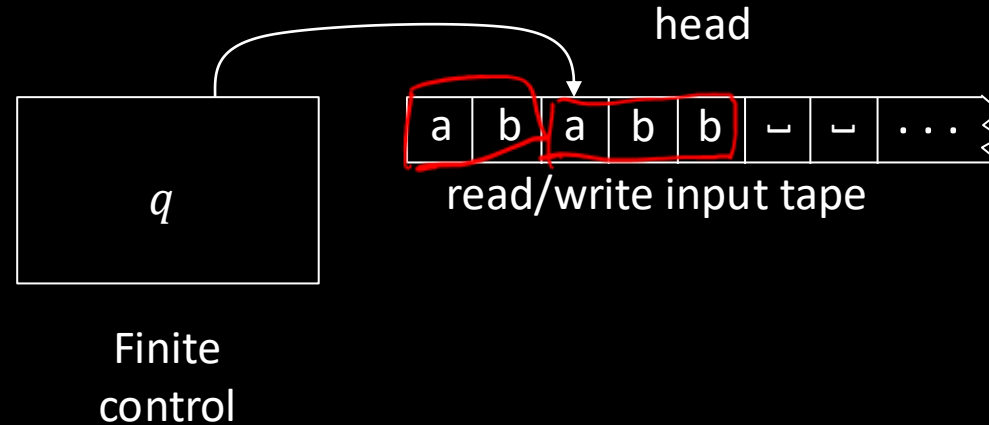
q_0 : w empty or w does not contain 11 and ends with 0

Turing Machine Configurations

The configuration of a TM is a “snapshot of its execution at a point in time”:

- Current state
- Current contents of the tape
- Current location of the tape head

Example



This configuration is represented using the notation $abqabb$

Notation: If applying δ to config C yields config C' write $C \Rightarrow C'$

Turing Machine Configurations

Examples

$uqbv \Rightarrow uctv$ if $\delta(q, b) = (t, c, R)$

$uqbv \Rightarrow tucv$ if $\delta(q, b) = (t, c, L)$

$qbuv \Rightarrow tcuv$ if $\delta(q, b) = (t, c, L)$ (tape head can't move to left if it's already at the start)

Turing Machine Computation Path

Start configuration of M on input w is $C_1 = q_0w$

Let C_k be the configuration after applying the transition function $k - 1$ times.

Defn. M halts on w if after finite number of transitions from C_1 , we obtain a configuration whose state is q_{acc} (M accepts w) or q_{rej} (M rejects w).

Defn. M does not halt (or loops) on w if for every k , the state of configuration C_k is neither q_{acc} nor q_{rej} .

Defn: TM M is a decider if M halts on all inputs and a recognizer otherwise.

IMPORTANT: For every string, a finite automata either accepts or rejects, but a TM can loop.

TM Recognizers and Deciders

Let M be a TM. Then $L(M) = \{w \mid M \text{ accepts } w\}$.

Say that M recognizes A if $A = L(M)$

Defn: A is Turing-recognizable if $A = L(M)$ for some TM M (aka semi-decidable).

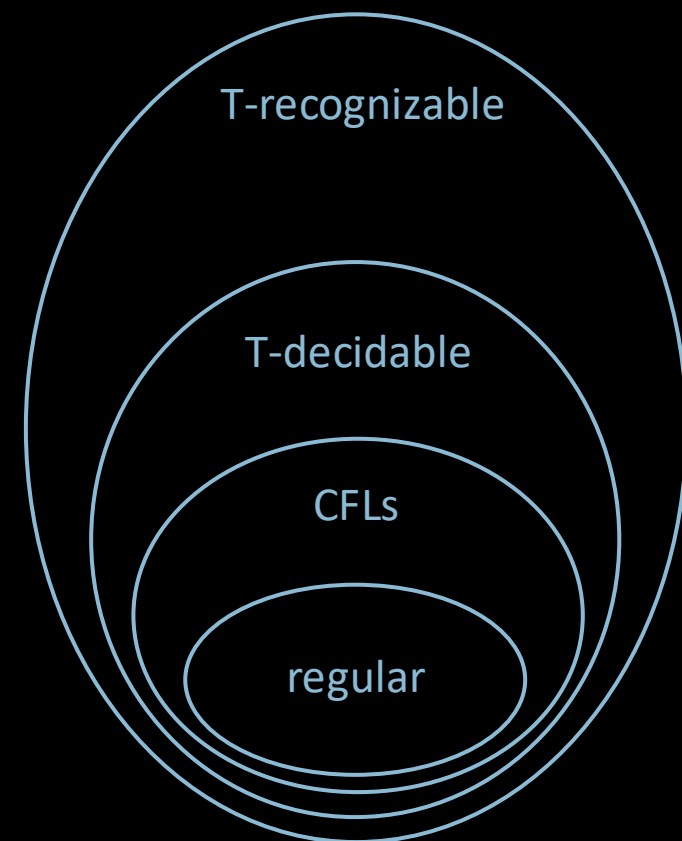
Say that M decides A if $A = L(M)$ and M is a decider.

Defn: A is Turing-decidable if $A = L(M)$ for some TM decider M .

IMPORTANT: A recognizer of L can loop on any number of strings not in L BUT a decider must halt and reject on every string not in L

Satisfiability of Prop Logic: Decidable

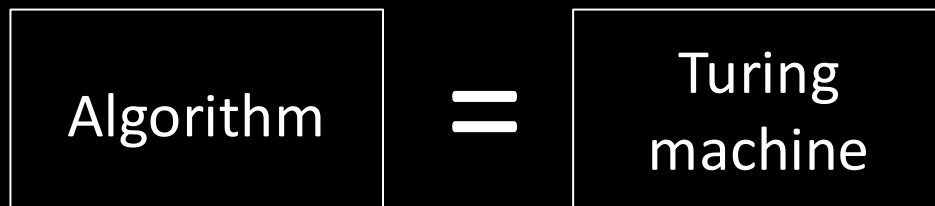
Satisfiability of First-Order Logic: Recognizable, undecidable



Church-Turing Thesis ~1936

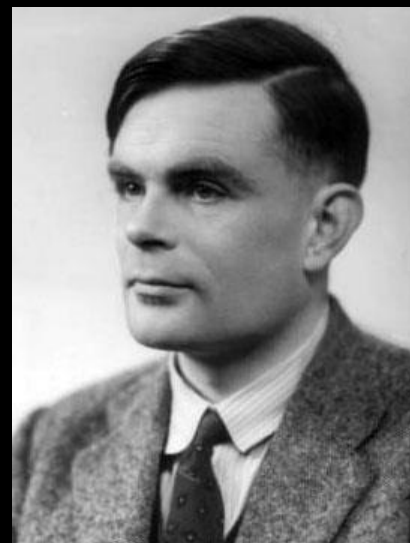


Alonzo Church
1903–1995



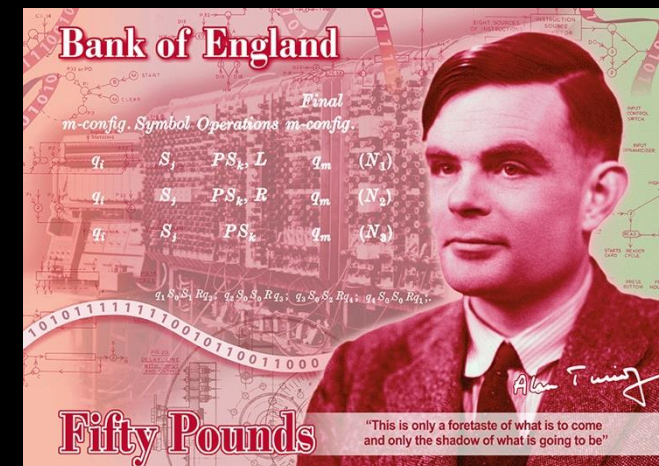
Intuitive Formal

Every “reasonable” model
of unrestricted computation is
equivalent to Turing machines:
 λ -calculus, rewriting systems, random
access machine, your favorite
programming language, ...

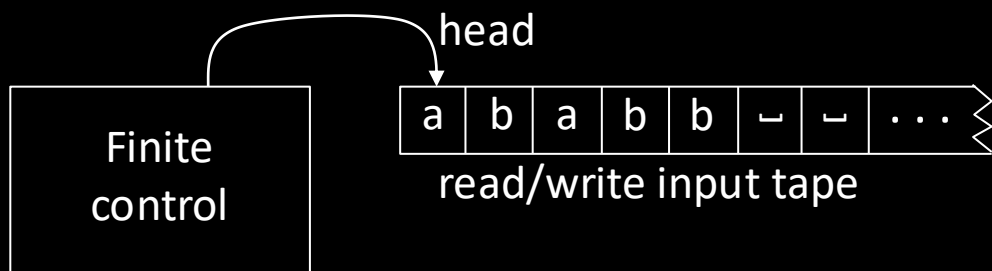


Alan Turing
1912–1954

Big impact on mathematics.



Turing machine model – review



On input w a TM M may halt (enter q_{acc} or q_{rej}) or loop (run forever).

So M has 3 possible outcomes for each input w :

1. Accept w (enter q_{acc})
2. Reject w by halting (enter q_{rej})
3. Loop (running forever)

Satisfiability of Prop Logic: Decidable

Satisfiability of First-Order Logic: Recognizable, undecidable

A is T-recognizable if $A = L(M)$ for some TM M .

A is T-decidable if $A = L(M)$ for some TM decider M .
halts on all inputs ↗

Turing machines model general-purpose computation.

Q: Why pick this model?

A: C-T Thesis:

All reasonable models are equivalent in power.

Virtues of TMs: simplicity, familiarity (similar to calculation using pencil and paper).

Turing machine model – robustness

- We will next see that adding to TM model does not increase computational power
- Examples:
 - Multi-tape Turing machines
 - Nondeterministic Turing machines
- Classic TM can simulate them